Projekt nr 3 - Wykorzystanie biblioteki PyGAD do algorytmów genetycznych w języku Python

Optymalizacja funkcji z wykorzystaniem biblioteki PyGAD.

1. Wykorzystamy szkielet projektu z przykładu z wykładu example_02.py.
2. Musimy ustalić zakres naszych parametrów w przedziale [0,1] konfigurując odpowiednio init_range_low na 0 oraz init_range_high na 2 i gene_type = int.
   Umożliwi nam to wygenerowanie osobnika, który będzie się składał z 0 i 1.
   Musimy pamiętać gdzieś w kodzie, że jeśli nasz osobnik ma np. 60 bitów – to pierwsze 20 bitów to pierwsza zmienna, a drugie 20 bitów to druga zmienna, a trzecie 20 bitów to trzecia zmienna.
   Liczba bitów i co za tym idzie liczba zmiennych optymalizowanej funkcji powinna być możliwa do konfiguracji.
   Liczbę bitów konfigurujemy w polu num_genes.
3. Przygotuj funkcję celu, którą będziesz optymalizować:

```
def fitnessFunction(individual):
    #tutaj rozkoduj binarnego osobnika! Napisz funkcje decodeInd
    ind = decodeInd(individual)
    result = (ind[0] + 2* ind[1] - 7)**2 + (2* ind[0] + ind[1] -5)**2
    return result,
```

Niech to będą funkcje realizowane w ramach projektu nr 1 i 2.

4. Przetestujmy wybrane metody selekcji z biblioteki PyGAD: selekcje turniejową (tournament) , koło ruletki (rws), selekcje losową (random).

5. W kolejnym kroku wybierzemy algorytm krzyżowania: krzyżowanie jednopunktowe (single_point), krzyżowanie dwupunktowe (two_points), krzyżowanie jednorodne (uniform).

6. Przetestujmy 2 algorytmy mutacji: losową (random) i zamiana indeksów (swap).
7. Wykonajmy stosowne eksperymenty odpowiednio konfigurując naszą główną klasę z PyGAD (przykład z example_02.py)

```
8. ga_instance = pygad.GA(num_generations=num_generations,
                          sol_per_pop=sol_per_pop,
                          num_parents_mating=num_parents_mating,
                          num_genes=num_genes,
                          fitness_func=fitness_func,
                          init_range_low=0,
                          init_range_high=2,
                          gene_type = int,
                          mutation_num_genes=mutation_num_genes,
                          parent_selection_type=parent_selection_type,
                          crossover_type=crossover_type,
                          mutation_type=mutation_type,
                          keep_elitism= 1,
                          K_tournament=3,
                          random_mutation_max_val=32.768,
                          random_mutation_min_val=-32.768,
```

```
            logger=logger,
            on_generation=on_generation,
            parallel_processing=['thread', 4])
```

9. Przygotujmy wariant alternatywny z reprezentacją rzeczywistą.

Zmieniamy init_range_low na dolny przedział poszukiwań oraz init_range_high na górny przedział i gene_type = float

Biblioteka PyGAD nie ma zaimplementowanych algorytmów krzyżowania dla reprezentacji rzeczywistej. Wykorzystaj te algorytmy które zaimplementowałeś w projekcie nr 2 i dostosuj je tak by wykorzystać je w bibliotece PyGAD.

Zwróć uwagę jak autorzy PyGAD implementują krzyżowanie

```
def single_point_crossover(self, parents, offspring_size):

    """
    Applies the single-point crossover. It selects a point randomly at
which crossover takes place between the pairs of parents.
    It accepts 2 parameters:
        -parents: The parents to mate for producing the offspring.
        -offspring_size: The size of the offspring to produce.
    It returns an array the produced offspring.
    """

    if self.gene_type_single == True:
        offspring = numpy.empty(offspring_size, dtype=self.gene_type[0])
    else:
        offspring = numpy.empty(offspring_size, dtype=object)

    for k in range(offspring_size[0]):
        # The point at which crossover takes place between two parents.
Usually, it is at the center.
        crossover_point = numpy.random.randint(low=0,
high=parents.shape[1], size=1)[0]

        if not (self.crossover_probability is None):
            probs = numpy.random.random(size=parents.shape[0])
            indices = numpy.where(probs <= self.crossover_probability)[0]

            # If no parent satisfied the probability, no crossover is
applied and a parent is selected.
            if len(indices) == 0:
                offspring[k, :] = parents[k % parents.shape[0], :]
                continue
            elif len(indices) == 1:
                parent1_idx = indices[0]
                parent2_idx = parent1_idx
            else:
                indices = random.sample(list(set(indices)), 2)
                parent1_idx = indices[0]
                parent2_idx = indices[1]
        else:
            # Index of the first parent to mate.
            parent1_idx = k % parents.shape[0]
            # Index of the second parent to mate.
            parent2_idx = (k+1) % parents.shape[0]

        # The new offspring has its first half of its genes from the first
```

```
parent.
        offspring[k, 0:crossover_point] = parents[parent1_idx,
0:crossover_point]
        # The new offspring has its second half of its genes from the
second parent.
        offspring[k, crossover_point:] = parents[parent2_idx,
crossover_point:]

        if self.allow_duplicate_genes == False:
            if self.gene_space is None:
                offspring[k], _, _ =
self.solve_duplicate_genes_randomly(solution=offspring[k],

min_val=self.random_mutation_min_val,

max_val=self.random_mutation_max_val,

mutation_by_replacement=self.mutation_by_replacement,

gene_type=self.gene_type,

num_trials=10)
            else:
                offspring[k], _, _ =
self.solve_duplicate_genes_by_space(solution=offspring[k],

gene_type=self.gene_type,

num_trials=10)


    return offspring
```

Przykładowa implementacja własnego algorytmu

```
def crossover_func(parents, offspring_size, ga_instance):
    offspring = []
    idx = 0
    while len(offspring) != offspring_size[0]:
        parent1 = parents[idx % parents.shape[0], :].copy()
        parent2 = parents[(idx + 1) % parents.shape[0], :].copy()

        random_split_point = numpy.random.choice(range(offspring_size[1]))

        parent1[random_split_point:] = parent2[random_split_point:]

        offspring.append(parent1)

        idx += 1
    return numpy.array(offspring)
```

Funkcja krzyżująca zwraca gotową populację po procesie krzyżowania.

Zaimplementujmy dodatkowo mutację Gaussa bazując na

```
def mutation_func(offspring, ga_instance):

    for chromosome_idx in range(offspring.shape[0]):
        random_gene_idx = numpy.random.choice(range(offspring.shape[1]))

        offspring[chromosome_idx, random_gene_idx] += numpy.random.random()
    return offspring
```

Zadania do wykonania

1) Dokonaj optymalizacji twojej funkcji z projektu nr 1 oraz 2.

Wykorzystaj gotowe biblioteki zawierające implementacje funkcji

```
import benchmark_functions as bf
from opfunu import cec_based

#benchmark_functions
#pip install benchmark_functions
func = bf.Hyperellipsoid(n_dimensions=10)

print(func.suggested_bounds())
print(func.minimum())

#cec
#pip install opfunu

func = cec_based.cec2014.F32014(ndim=10)
print(func.bounds)
print(func.x_global)
print(func.f_global)
```

2) Wykorzystaj reprezentacje binarną jak i rzeczywistą.
3) Program wykonaj w wersji console application. Nie jest wymagane przygotowanie menu do konfiguracji programu.
4) Przygotuj sprawozdanie podobne do sprawozdań z projektów nr 1 oraz nr 2.